

Creating Your First Data Connection

in Palantir Foundry

Rajesh Pasham

S3 Bucket

Relational DB

REST API

Agenda

01

Introduction & Setup

Folder creation, Foundry overview

02

Data Connection Basics

Agents vs Direct Connection, architecture

03

S3 Connection

Connect to AWS S3, configure sync, import data

04

Relational Database

PostgreSQL connection, SSL certificates

05

REST API Connection

API source, Code Repository, Python transforms

Introduction

What You Will Learn

- Connect to Palantir-provided data sources
- Set up the three most common connection types
- Configure egress policies for secure access
- Sync external data into Foundry datasets
- Manage data pipelines via an intuitive interface

Data Sources Covered

AWS S3

Cloud object storage for flat files

PostgreSQL

Relational database (JDBC protocol)

REST API

HTTP endpoints via Code Repository



No need to provide your own datasources — use Palantir-provided training sources.

Agents vs Direct Connection



Agent Connection

- Downloadable program installed in your org network
- Required for private networks & on-premises systems
- Communicates via encrypted outbound requests
- Requires provisioning, configuration & maintenance
- Best for: internal databases, on-prem file stores

VS



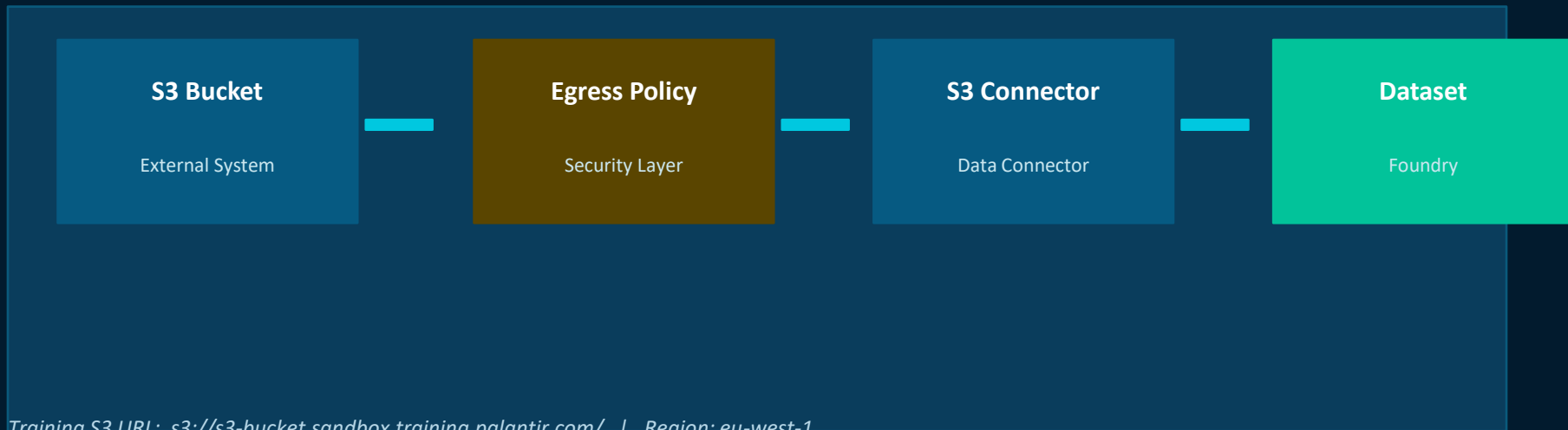
Direct Connection

- For data sources accessible over the Internet
- Supports REST APIs, SFTP, cloud storage accounts
- No agent to provision, configure, or manage
- Automatically managed — excellent uptime
- Best for: S3, PostgreSQL (cloud), REST APIs ✓

Data Connection:

AWS S3 Bucket

Connect Foundry to an AWS S3 bucket and sync flat files into Foundry datasets



S3 Connection — Step by Step

1

Open Data Connection App

Applications → Data Connection → New source

2

Select S3 Source Type

Choose S3 connector → Click Continue

3

Choose Direct Connection

S3 is publicly accessible over the Internet

4

Name & Save Source

"Data Connection Training - S3 (your_name)" → select project

5

Enter Connection Details

URL, region, access key ID, secret access key

6

Configure Egress Policy

Import existing or create new policy to whitelist the source

7

Create Sync & Build

Explore source → select file → Create sync → run build

S3 — Connection Credentials

Connection Settings

URL	s3://s3-bucket.sandbox.training.palantir.com/
Region	eu-west-1
Auth Method	Access key and secret
Access Key ID	AKIAWNSQVQU4RPCWEESQ
Secret Access Key	tWpDRP2QcCU1...m98 (encrypted)

Egress Policy

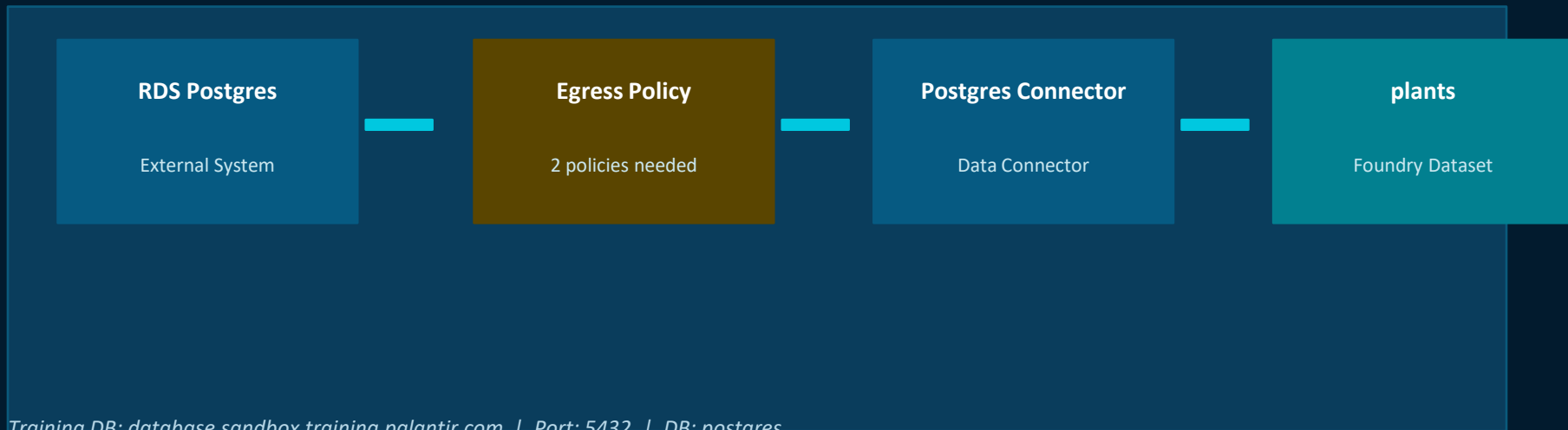
- Whitelists the data source for secure Foundry access
- One-time setup — can be reused by later users
- Foundry auto-suggests the policy based on URL
- Import existing OR create new policy
- Address: s3-bucket.sandbox.training.palantir.com.s3.eu-west-1.amazonaws.com
- Port: 443

✓ After sync completes, open dataset → Apply a schema → data is ready for use in Foundry pipelines.

Data Connection:

Relational Database

Connect Foundry to a PostgreSQL database using the JDBC protocol with SSL encryption



PostgreSQL — Configuration Details

Connection Settings

Host Type	Hostname
Hostname	database.sandbox.training.palantir.com
Port	5432
Database	postgres
Username	postgres_user
Password	speakefriendandenterdatabase

Two Egress Policies Required

- Policy 1 — DNS: database.sandbox.training.palantir.com (Port 5432)
- Policy 2 — IP: 52.17.120.168 (Port 5432)

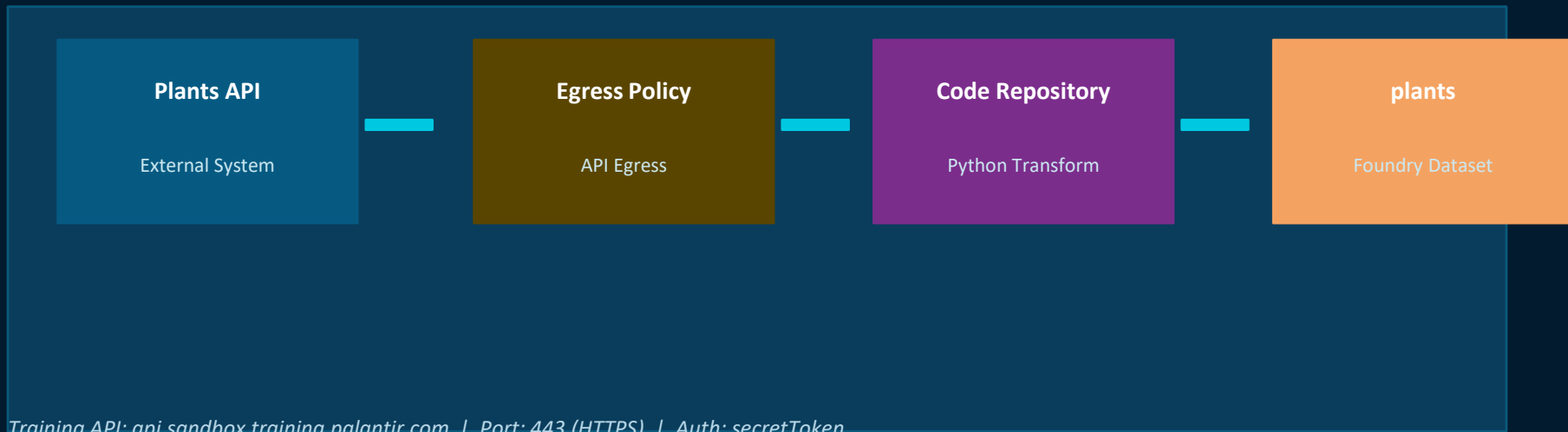
SSL Certificate Required

- PostgreSQL requires SSL for encrypted connection
- Download server-certificate.txt from lesson
- Alias: server.pem
- SSL Mode: require
- S3 does NOT need certificates (AWS handles it)

Data Connection:

REST API

Connect to HTTP endpoints using Foundry Code Repositories and Python transforms



REST API — Setup Workflow

1

Create REST API Source

Data Connection → New Source → REST API → Foundry worker

2

Configure Domain & Auth

URL: api.sandbox.training.palantir.com, Port 443, auth=None, secret=secretToken

3

Set Egress Policy

Import or create policy for api.sandbox.training.palantir.com:443

4

Enable Code Import

Allow source import into code repositories, set API name (camelCase)

5

Create Code Repository

Applications → Code Repositories → New Repo → Pipelines > Python → Spark Transform

6

Install Library

Libraries panel → search 'transforms-external-systems' → Add and install

7

Write Python Transform

Import source, call GET /secretPlants with auth header, write to output dataset

8

Commit & Build

Commit changes → Build → wait for successful run → view dataset

REST API — Python Transform Code

```
from transforms.api import transform_df, Output
from transforms.external.systems import external_systems, Source

@external_systems(rest_api_source=Source("<<SOURCE RID>>"))
@transform_df(Output("<<OUTPUT_PATH>>"))
def compute(rest_api_source, ctx):
    url = rest_api_source.get_https_connection().url + "/secretPlants"
    auth_token = rest_api_source.get_secret("additionalSecretSecret")
    client = rest_api_source.get_https_connection().get_client()
    headers = { "Authorization" : auth_token }
    response = client.get(url, headers=headers)
    response.raise_for_status()
    json_content = response.json()
    return ctx.spark_session.createDataFrame(json_content)
```

Replace <<SOURCE RID>> with your source RID from the External Systems tab, and <<OUTPUT_PATH>> with your dataset path.

Key Concepts Summary

Data Connection

Foundry app that synchronizes data from external systems into Foundry datasets with a frontend interface.

Egress Policy

Network security policy that whitelists external data source addresses. Required for all Direct Connections.

Direct Connection

Used for Internet-accessible sources. No agent needed. Automatically managed by Foundry.

Agent Connection

Used for private networks & on-premises systems. Requires installing an agent within your org network.

Sync & Dataset Build

A Sync defines what data to bring in; a Build executes the import. Builds update dataset contents.

Code Repository (REST)

Python transforms in Foundry Code Repositories pull data from REST APIs using transforms-external-systems.

You've successfully learned how to:

- ✓ Set up data connections in Palantir Foundry
- ✓ Connect to AWS S3 cloud object storage
- ✓ Connect to PostgreSQL relational databases
- ✓ Connect to REST APIs via Code Repositories
- ✓ Configure egress policies for secure access

Rajesh Pasham